

Computer Science 1015F ~ 2022 ~ Notes to Students

Course Description

Computer Science 1015F and 1016S together constitute a complete Computer Science curriculum for first year students, offering an introduction to the development of algorithms and design of computer programs, along with other selected topics in Computer Science. CSC1015F is offered in the first semester and CSC1016S is offered in the second semester. The course is offered in a blended learning format. Student contact time is in a single face-to-face lecture per week as well as a practical session aimed at reinforcing the principles introduced in the online lectures and giving students time to do exercises under the supervision of tutors.

Prerequisites

Mathematics at NSC level 6 or better (70%).

No prior knowledge of computer programming is necessary!

Staff

Course Convener:	Mr. Aslam Safla	aslam@cs.uct.ac.za
Online Lecturers:	Prof. James Gain Prof. H. Suleman Mr. Aslam Safla	jgain@cs.uct.ac.za hussain@cs.uct.ac.za aslam@cs.uct.ac.za
Teaching Assistants (TAs):	(will be announced on Vula)	
Tutors:	(will be announced on Vula)	

Tentative Schedule of Lectures and Practical Work

	M	T	W	T	F	Ptest	Practical
14-Feb	Intro	Basics	Basics	Basics			Orientation
21-Feb	Basics	Selection	Selection	Selection			1 - Introduction
28-Feb	Selection	Iteration	Iteration	Iteration		Prac Test 1	2 - I/O, Control (if)
07-Mar	Iteration	Strings	Strings	Strings		Prac Test 1	3 - Control (if, while)
	No Lect. / Theory test 1						
14-Mar		No Lect.	Strings	Functions			4- Control (if, for, while)
21-Mar		Functions	Functions	Functions			5 - Strings
28-Mar							
04-Apr	Testing	Testing	Testing	Arrays		Prac Test 2	6 - Functions
11-Apr	Arrays	Arrays	Arrays	Recursion		Prac Test 2	7 - Testing
18-Apr		Recursion	Recursion	Recursion			8 - Arrays
25-Apr	Files	Files		Files			9 - Recursion
02-May		Files	S&S	S&S		Prac Test 3	10 - Files
	S&S / Theory test 2						
09-May		S&S	Number Systems	Number Systems		Prac Test 3	Consolidation
16-May	Number Systems	Number Systems	HW/SW	HW/SW			
23-May							

Textbook and Notes

There is *no prescribed textbook*. There is a departmental Python book that is structured roughly in order of the lectures' schedule of topics (see further below), available from:

http://www.cs.uct.ac.za/mit_notes/Python/

If you like hard copy books, then the following one is highly recommended:

Python Programming: An Introduction to Computer Science (second or third edition) *by John Zelle*, Franklin, Beedle and Associates, Inc.,

Class notes (copies of slides) may be available for selected sections and announced by the relevant lecturer(s). Electronic copies of lecture slides will be made available on Vula.

Vula

Vula (<http://vula.uct.ac.za>) is the university-wide online learning management system that gives you access to resources to assist in the learning process. The class website for all courses will be located on the Vula system.

Lecturers, TAs and tutors may be consulted through Vula – this is preferable since any questions that are answered may benefit other students as well. Vula is used for the submission of ALL practical assignments and practical tests and for providing students with marks for assignments and tests, and feedback where appropriate. All students will be expected to consult the website on a daily (Monday-Friday) basis for updates on lecture material, assignments, marks, hints, deadlines, etc.

Refrain from posting anything of the following nature anywhere on the website, as it may violate the university's Appropriate Use of Computer Facilities policy (see ICTS website), necessitating disciplinary and/or legal proceedings: sexist, racist or otherwise discriminatory comments, flame wars, trolling; segments of program code (other than something provided by the instructor); solutions to graded work (or part thereof) before or after submission; or illegal material.

Lectures

Lectures are held in the following venues:

	Venue
Period 4:	RW James 3A
Period 5:	JD LT1

Lectures are held from Monday-Friday every week. You are encouraged to attend any one of these lecture sessions per week.

Hotseat

The Computer Science HotSeat is run by senior tutors who can assist you to understand difficult concepts or work through problems you have encountered during lectures, assignments or tests. The HotSeat tutor is available on level 2A in CS2A.2 in the Computer Science building (the mentor room next to the Senior lab). A schedule of available times will be posted on Vula once finalised.

Tests

There will be 2 closed-book 40 minute theory tests. Each test will run in two or three sessions; venue to be confirmed.

Test 1: Monday, 14 March, 11am or 12 noon (details will follow)

Test 2: Monday, 9 May, 6pm or 7 pm (details will follow)

There will be 3 open-book 40 minute practical tests as part of the laboratory sessions. Each practical test will be offered 2 times; you may write one or both of them (so, only once a week) and the maximum mark obtained will be used for each test. You have to do the test in the practical session you signed up for.

See the schedule for detailed test dates. These tests contribute to your DP mark and your final mark.

Laboratory Sessions

Laboratory sessions are held either in one of the Scilabs or in a lecture room. You must attend one 2-hour session each week, where you may work on the current practical assignment and discuss any general issues related to practical work with the tutors who are available. Students will be given a weekly quiz during these sessions, and students write a practical test (as described above) if one is scheduled for that week.

It is your responsibility to ensure that your laboratory session (on Vula) that fits in with your timetable. If you discover that all slots that suit you best are full already, *it is your responsibility to find a student with whom you can arrange a swap as soon as possible.*

Questions and Submission

All questions for assignments, along with all related files, will be available on the class website on Vula. Practical assignments must be submitted electronically via Vula ONLY. The online submission system used to receive your assignments will provide the official timestamp used to determine whether a program is on time. Marks will be deducted automatically for automatically-marked assignments that are submitted late.

Marking

Most assignments will be marked automatically based on test cases and the marks will be uploaded to Vula. Tutors will mark randomly-chosen practical assignments during the semester.

Equipment and Programming Language

All programming will be done in Python v3 unless otherwise stated.

It is the responsibility of the student to submit a program that will successfully execute on the specified platform. Any student who works on their own equipment must ensure that all assignments will execute on the university equipment before submission – no discussion will be entered into after submission.

Computing facilities are available for use in the Scilabs that are located in the Computer Science building (Scilab A and B), RW James (Scilab C) and P D Hahn (Scilab D).

It is ALWAYS the student's responsibility to ensure that adequate backup copies are made of all work in progress and all work already completed. Loss of data or programs is not an acceptable excuse for non-submission or late submission of assignments.

Attendance, Exemptions, Extensions for Practical Work

Exemption and extensions of deadlines for practicals and tutorials will only be considered on medical or compassionate grounds. You must send an email to the course convenor explaining the circumstances around the request. Make sure that your full name, student number, course code and the reason for your request are clearly indicated in the email. Medical grounds must be supported by a valid medical certificate - see below.

Compassionate grounds will normally require a letter of support. See the course convenor regarding compassionate grounds. Do this before the deadline in question, or if that is not possible, immediately on your return to UCT. It is vital to do this immediately as late applications after the deadlines have passed will not be considered. The lecturers, TA and tutors cannot grant extensions and exemptions - do not ask them.

Plagiarism

Refer to attached document for the departmental plagiarism policy. *This policy will be strictly enforced.*

All assignments, tests and examinations are done individually – there is NO group work allowed in this course. It is acceptable to discuss the questions for assignments with peers but not the specific details of the solutions, nor engage in “pair programming”. When in doubt, speak to a tutor or TA.

Students are required to sign and submit a form (on the last page) verifying that they have read and understood the contents of this policy before commencing any form of assessed work.

There will be randomised code plagiarism checking.

DP Requirement

A student is granted DP status (and therewith may write the exam) in CSC1015F if the following condition is met:

- $(3/5 * \text{Assignment average} + 2/5 * \text{Practical test average}) \geq 45\%$

where Assignment Average = $0.90 * \text{Weekly Assignment Average} + 0.10 * \text{Quiz Average}$

Final Examination

The examination timetable will be published sufficiently in advance of the final examination on university notice boards. It is the student's responsibility to take note of the correct time and place for the examinations.

All examinations will be cumulative, closed-book and closed-notes (i.e., you may not bring your notes or textbooks into the examination room), and 2 hours in duration.

A final mark in CSC1015F will be calculated as follows:

$$\text{Final} = 0.15 * \text{Assignment average} + 0.15 * \text{Test average} + 0.10 * \text{Practical test average} + 0.60 * \text{Exam}$$

In order to pass, ALL of the following requirements MUST be met:

- Final $\geq 50\%$
- $(3/5 * \text{Assignment average} + 2/5 * \text{Practical test average}) \geq 45\%$
- $(1/5 * \text{Test average} + 4/5 * \text{Exam}) \geq 45\%$

All students who pass CSC1015F are eligible to continue with CSC1016S, as are those students who are granted a supplementary exam.

Supplementary Examinations

Students who do not pass but obtain a mark of 45-49 may be awarded a supplementary examination; this is indicated either with xS (with $45 \leq x \leq 49$) or OSS ['failed theory subminimum'] in PeopleSoft. These are written in **July, usually in the first week of the 2nd semester**. A Vula announcement will be made regarding the exact time and date of the supplementary exam.

The final mark will be calculated as above, but then with the supplementary exam mark. The system then converts this into a "UP" unqualified pass or "UF" unqualified fail if you had a plain supp and into the 'June grade' if you had an OSS with the June exam.

Note that there are no other examination opportunities for this course. Please note that deferred exams do not allow the option of a subsequent supplementary exam: if you fail a deferred exam, you have failed the course.

Grade Allocation

1 = 75-100; 2+ = 70-74; 2- = 60-69; 3 = 50-59; F = 0-49.

Information Dissemination and Communication

Medical Notes and Short Leave

All students are expected to do all assigned work. If a student falls ill, they must submit a medical certificate to the course convenor as soon as possible.

A valid medical certificate must be signed by a qualified medical doctor and clearly state the days on which you were medically unfit to work. Please submit the original medical certificate to Ms T Jenneker (Room 317) and email the course convenor noting that you have submitted a medical certificate. If you are unable to deliver the medical in person, you may email a scanned version to sick-csc1015f@cs.uct.ac.za. Once you are able to do so, you must hand in the original as instructed above.

In other/exceptional circumstances, where a student would like to be away from their studies for a short and defined period of time, the Short Leave Application form (under Vula resources) must be filled out and submitted to the course convenor for recommendation and HoD for approval.

Appeals Procedure

If a student feels that their marks are incorrect for any piece of marked work, they must first approach the tutor or TA responsible for that test or assignment. Then, if need be, they may appeal to the course convenor. Finally, they may appeal to the HoD.

Queries

Any queries about the *content* of the lectures MUST be directed to the lecturer teaching that section.

Any queries about *marks* or *marking* of practical assignments must be directed to your tutor or TA.

All marked work (whether in paper or electronic format) must be kept until the end of the semester. In general, queries about marks MUST be made within a week of marked work being returned. No queries about any marks will be entertained after the final examination.

Any queries about the *administration* of the course must be directed to the TA.

The course convenor must ONLY be contacted as a last resort, unless otherwise indicated.

Disability

If any student needs special accommodation because of a disability, please contact the course convenor during the first week of classes.

Syllabus

Corresponding chapter numbers in the recommended textbook are indicated in parentheses.

- Introduction to Computer Science (Ch. 1): What is Computer Science, Applications of Computing, History of Computing, Computer Hardware (Machine Architecture), Computer Software (System Software, Applications), Algorithms, Programming Languages
- Introduction to Python Syntax (Ch. 2.1-2.5; 3): Basic syntax, variables, operators, comments, expressions, output
- Conditionals (Ch. 7): Boolean expressions and logical conditions, If statements, nested ifs, if-else, if ladders,
- Loops (Ch. 8): for, while, nested loops
- Strings and Input (Ch. 5)
- Functions (Ch. 6): parameters, return values
- Testing: debugging, equivalence classes
- Arrays (Ch. 5; 11): lists, dictionaries, sets, multi-dimensional arrays
- Recursion (Ch. 13.2)
- Sorting and Searching (Ch. 13.1; 13.3)
- File I/O (Ch. 5.9): text files, exceptions
- Number Systems: Machine representations of data, Binary operations, Boolean algebra

Science Extended Degree Programme

The Computer Science Department runs academic development courses – CSC1010H and CSC1011H – that are part of the Extended Degree Programme in Science (EDP). CSC1010H runs over approximately an entire year and covers the syllabus of CSC1015F at a somewhat slower pace and in greater depth. In the first semester of the following year of study, CSC1011H covers additional material to assist students in their transition to CSC1016S, which forms the second semester module of CSC1011H. Note that each of these EDP courses counts only as a half credit.

Students who do not perform adequately in the first test in CSC1015F will be invited to join CSC1010H at the beginning of the second quarter, on condition that the student has a good record of submitting practical assignments and there is sufficient space in the CSC1010H course. Any student who feels that they may benefit by joining CSC1010H should consult Aslam Safla (Course convenor) or Gary Steward (EDP Coordinator) before the end of the first quarter.

Extra Problems: the CSC1015 Challenge

This course is about problem solving, but for some of you, the exercises and assignment problems may be not enough, be this because you already may know how to program or you would like to see more real-life problems that can be solved using a computer. Therefore, we have decided to add a challenge. This is not for marks, but you can gain (small) prizes with it. You should *only* attempt this *after* you've *successfully* completed *all* your homework and assignments and

tests. Note also that tutors and lecturers will *not* assist you in solving the problem—this is you honing your problem-solving and programming skills.

The setting and rules are as follows.

- There are 12 problems to solve, which are of varying level of difficulty. The points you can score is an indication of difficulty and/or amount of code needed for the solution (more points = harder or more work). The topics of the problems range from a few problems for fun to realistic ones (e.g., managing train tickets on the metrorail, an app for fair goods exchanges, automating conveyor belt operations, optimising a tourism destination).
- The programming techniques you will need to implement the solution are to a greater or lesser extent covered in the course (arrays, if-else, sorting etc.). The skills and techniques needed to solve the problems match the order of the course's syllabus for at least several problems, but not all.
- As with assignments and prac tests, the automatic marker will be used. However, the only information in the report is “wrong answer” or “accepted”; that is, unlike with the assignments, you do not get to see the test cases. The marking is an all-or-nothing: either all test cases complete successfully and you will get the points, or not and have one penalty point deducted.
- The system only accepts solutions written in Python.
- It is not a weekly race against the clock, but a final deadline there is: you have until 8 May to submit solutions.
- There will be essentially 3 scoreboards, merged into one in the leaderboard interface: 1) general leaderboard based on points accumulated having solved one or more problems, 2) a first-to-solve-the-problem scoreboard (colour-coded in green), and 3) a first *new* student to solve a problem scoreboard (colour-coded maroon). On the latter, e.g., Joe Soap is the first to solve Problems A, B, and C, in that order, then that will appear as such in the scoreboard in green. Then Joanne Soap solves Problem C, which causes the scoreboard to update with a maroon-solved. Other solved problems will have a blue colour. Any other solved problem is colour-coded with blue. When a potential solution is submitted but was not fully right yet, then it will turn up as a pink box.
- To keep some suspense, the scoreboard updates occur only once a day.
- The tutors will not help you solve these problems.
- Besides the honour to be on the scoreboards, there is a small prize for each first new student to solve a problem, and less small prizes at the end for the top-5 of the general leaderboard.

The Challenge will run on a separate Vula site.

Department of Computer Science University of Cape Town

Policy on Academic Dishonesty for Computer Program Submissions

Introduction

The University of Cape Town has well-defined policies on copying and plagiarism, which are contained in the general Rules for Students and set out in full on the UCT website at: <http://www.uct.ac.za/about/policies/>. The Department of Computer Science has set out the following guide and interpretation of these rules and policies as they apply to courses involving computer programming, and the use of the computer. The rules defined here are in line with the best procedures in other institutions and, in particular, are adapted from those on academic dishonesty at Oregon State University¹ and academic dishonesty and cheating at San Francisco State University².

The purpose of teaching and learning in the Computer Science Department is to enable you, the student, to understand the work that you have submitted. Thus, you are required to do your own work for an assignment in order to ensure that you understand it.

Academic Dishonesty for Computer Program Submissions

Academic dishonesty is EITHER:

- Copying and plagiarism of computer program submissions, where you present, as your own work, material produced by or in collaboration with others, without proper acknowledgment, and where implicitly or explicitly disallowed for the assigned work.

OR

- Permitting or assisting others to present your work as their own.

Guidelines for Determining Academic Dishonesty

The following guidelines are provided to help in determining if an incident of academic dishonesty has occurred. A member of staff may suspect a student of academic dishonesty if the student submits a program that is so similar to the program submitted by a present or past student, in whole or in part, in the course that the solutions may be converted to one another by a simple mechanical transformation. A member of staff may suspect a student of academic dishonesty, whether on a program or an examination, if the student cannot explain both the intricacies of his or her solution and the techniques and principles used to generate that solution.

In all circumstances, it is acceptable to discuss the meaning of assignments and general approaches and strategies for handling those assignments with other members of the Academic Community.

Any cooperation beyond that point, including shared pseudocode or flowcharts, shared code, or shared documentation, is only acceptable if specifically so permitted by the lecturer in written guidelines distributed to the entire class, either through printed handouts or on the course website or other official communications channel.

¹<http://cs.oregonstate.edu/acad/policies/dishonesty.htm>

²<http://cs.sfsu.edu/plagiarism.html>

Where a student working on an individual assignment includes the work of others (e.g., algorithms transcribed from a textbook), the student must clearly indicate in the code what part(s) are the work of others.

Penalty for Academic Dishonesty

In all cases where academic dishonesty is suspected, is tested, and is deemed to have occurred, no credit will be given for the assignment in question and a record will be kept in the Department of Computer Science. A report may also be submitted to the Vice-Chancellor's office for possible disciplinary action through the University Disciplinary Tribunal, which may include expulsion, suspension, or probation, as well as lesser sanctions. In all cases, students will have the right to be heard by the lecturer, the course convener and the Head of Department before any action is finalized.

Identifying Copying and Plagiarism in Computer Program Submissions

As the Department of Computer Science believes strongly that students should only get credit for their own work, it will take all necessary steps to identify any cases of copying and plagiarism of computer submissions. These could include:

- Submitting program files to plagiarism detection sites, either locally or internationally.
- Setting a test on the techniques used in a program.
- Requesting students to explain sections of their code.

If you have any doubt or if you have any questions, you should consult with your lecturer or course convenor as to whether or not your work with other students and programs prepared for submission are appropriate.

Examples of Academic Dishonesty

The following examples illustrate situations when Academic Dishonesty has and has not occurred. Please note that these lists are not comprehensive!

Academic Dishonesty has occurred:

- When a student turns in the work of another student and represents it as his or her own work.
- When a student includes another person's work in their own submission without setting out in full what part(s) are the work of others.
- When a student knowingly permits another to turn in his or her work, including by providing access to personal files through sharing of login credentials.
- When a student copies code from the work of another student.
- When a student deliberately transforms borrowed sections of code in order to disguise their origin.
- When several students collaborate on a project (whether or not group work is allowed) and fail to inform the lecturer of this.
- When a student steals, obtains solutions, or program samples from another student's output or personal files.
- When a student is unable to explain the working of a piece of code.

Academic Dishonesty has NOT occurred:

- When students have permission to collaborate on a project, and list all collaborators.

- When students receive advice from tutors, teaching assistants, or staff members involved in the course.
- When students share knowledge about syntax errors, coding tricks, or other language-specific information that makes programming easier, except where such techniques represent the core topics being tested in the assignment.
- When students engage in a general discussion of the nature of an assignment, the requirements for an assignment, or general implementation strategies.
- When students engage in discussion of course concepts or programming strategies in preparation for an assignment or examination.
- When students copy code and cite its source on assignments for which the lecturer allows inclusion of code other than the student's own.

Conclusion

The Computer Science Department believes that encouraging students to submit their own work and actively enforcing this policy will ensure that work done is appropriately rewarded. All students will benefit from the ensuing recognition of the high standard of graduates from our programme.

Document History

v1, 2003:

First version of policy

v2, February 2016:

Revision of policy to reflect practice and changes in operations